

Iterazione 3

Durante l'iterazione 3 ci siamo concentrati sullo sviluppo dell'UC1 e UC4 e dei relativi test.

UC1: Registrazione alla piattaforma

Nome caso d'uso	UC1: Registrazione alla piattaforma
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Visitatore
Parti interessate e interessi	Visitatore: vuole inserire i propri dati personali per creare un account e poter diventare uno Sportivo registrato.
Pre-condizioni	Il visitatore ha accesso all'applicazione/sito web.
Garanzie di successo	I dati del visitatore sono salvati nel sistema e il nuovo profilo Sportivo è creato correttamente.
Scenario principale di successo	1. Il Visitatore accede alla sezione di registrazione. 2. Il Sistema richiede i dati anagrafici e le credenziali (Nome, Cognome, Email, Password). 3. Il Visitatore inserisce i dati richiesti e conferma la registrazione. 4. Il Sistema verifica che l'indirizzo email inserito non sia già presente nel database. 5. Il Sistema registra il nuovo profilo, lo aggiunge all'elenco degli sportivi e mostra un messaggio di successo.

Estensioni

***a.** In qualsiasi momento il sistema fallisce e ha un arresto improvviso.

1. L'admin riavvia il software e ripristina lo stato precedente del sistema.

2. Il sistema ripristina lo stato.

4a. L'email inserita risulta già registrata nel sistema.

1. Il Sistema mostra un messaggio di errore ("Email già in uso").

2. Il Sistema chiede al Visitatore di inserire un'email diversa o di annullare l'operazione.

Modello di dominio:

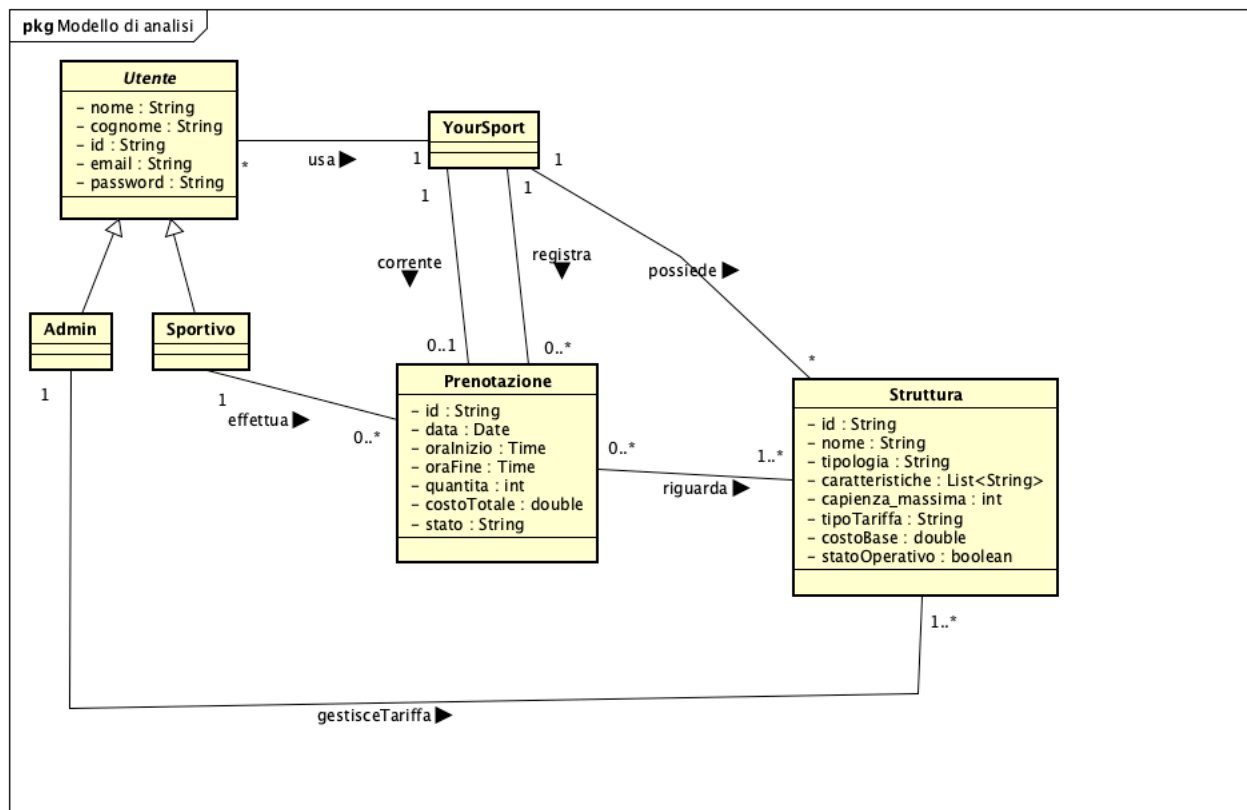
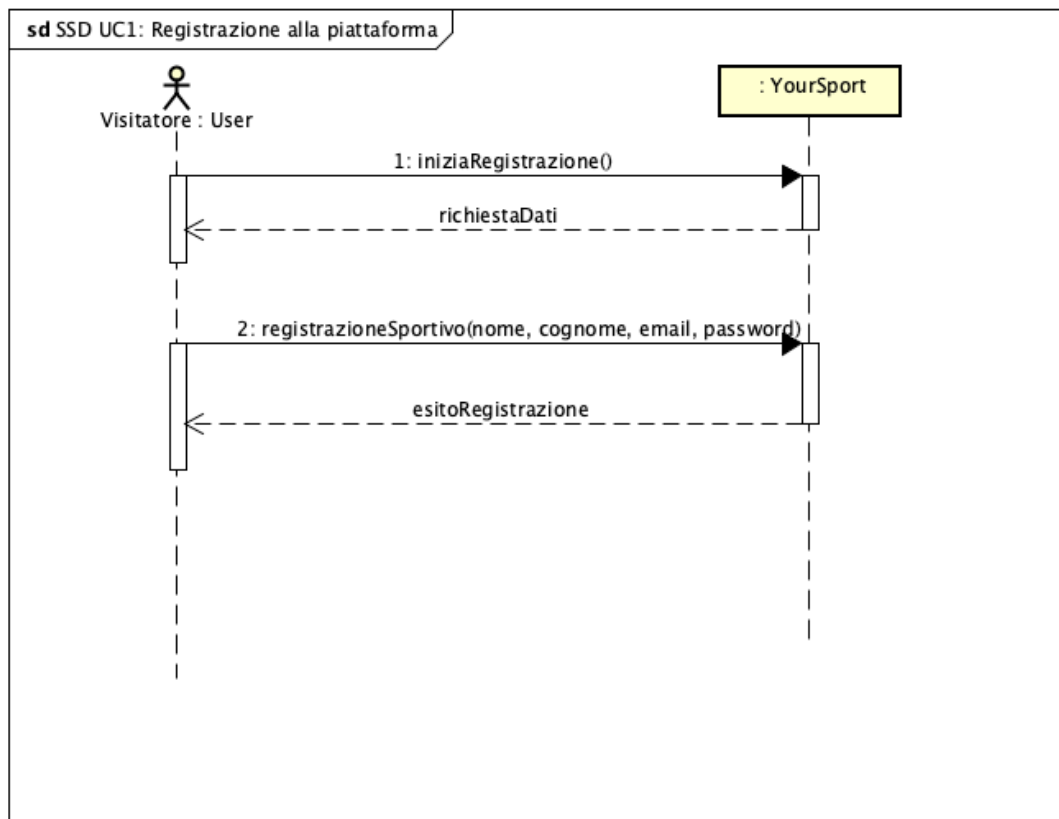


Diagramma di sequenza di sistema:



Contratti delle operazioni:

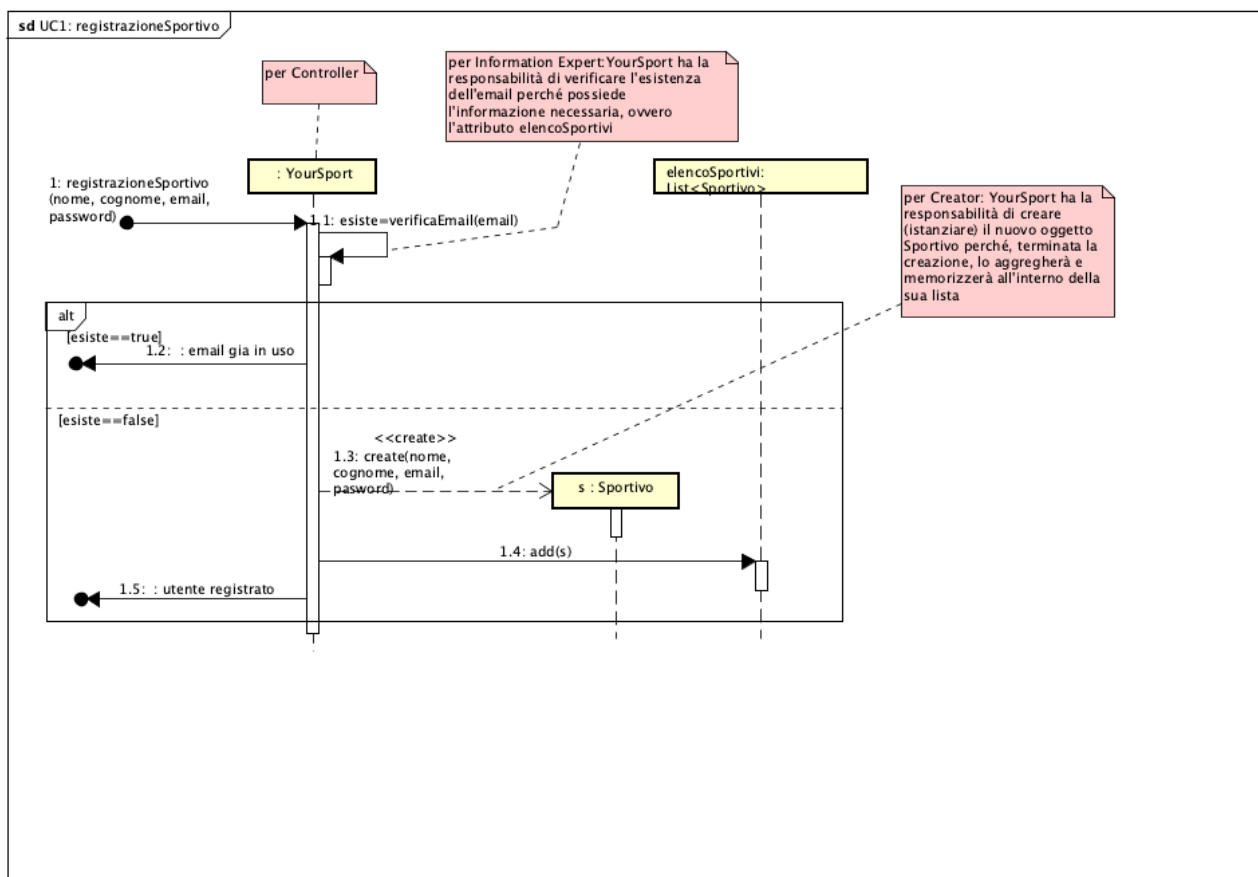
-UC1

Operazioni	registrazioneSportivo(nome: String, cognome: String, email: String, password: String)
Riferimenti	UC1: Registrazione alla piattaforma
Pre-condizioni	La stringa email non è presente in nessuna istanza di Sportivo già registrata nel sistema.

Post-condizioni	- È stata creata una nuova istanza s della classe Sportivo (<i>Creazione di istanza</i>). - Gli attributi nome, cognome, email e password dell'istanza s sono stati inizializzati con i valori dei parametri corrispondenti (<i>Modifica di attributi</i>). - L'istanza s è stata associata all'oggetto YourSport (inserita nel catalogo/elenco degli sportivi del sistema) (<i>Formazione di associazione</i>).
-----------------	---

Diagrammi di sequenza:

-registrazioneSportivo



Testing:

Test 1: Registrazione Nuovo Utente con Successo (Happy Path)

- Descrizione:** Simuliamo un nuovo utente non ancora presente nel sistema che inserisce i propri dati (nome, cognome, email libera e password) per creare un account.

- **Scopo:** Verificare che il sistema elabori correttamente il flusso principale dell'UC1, istanziando il nuovo oggetto Sportivo (Pattern Creator) e aggregandolo alla collezione degli utenti del sistema.
- **Realizzazione:** Il test chiama il metodo registrazioneSportivo() passando dati del tutto nuovi. Si verifica tramite asserzione (assertEquals) che il sistema restituisca la stringa "utente registrato". Come step finale, si esegue una verifica strutturale (tramite assertTrue invocando il metodo login) per accertarsi che il nuovo utente esista effettivamente in memoria e possa autenticarsi.

Test 2: Registrazione Bloccata per Email Duplicata (Flusso Alternativo)

- **Descrizione:** Simuliamo un utente che tenta di registrarsi utilizzando un indirizzo email già associato a un account esistente nel sistema (nel nostro caso, la mail di default "mario@email.it").
- **Scopo:** Assicurarsi che il sistema rispetti la pre-condizione e il blocco *ALT* del Diagramma di Sequenza, impedendo la creazione di account cloni e mantenendo la coerenza dei dati.
- **Realizzazione:** Il test chiama registrazioneSportivo() passando un indirizzo email già noto al sistema. Tramite assertEquals si verifica che il sistema intercetti la collisione (grazie al metodo verificaEmail) e restituisca esattamente il messaggio d'errore previsto: "email già in uso", interrompendo la creazione dell'oggetto.

UC4: Gestione delle proprie prenotazioni

Nome caso d'uso	UC4: Gestione delle proprie prenotazioni
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Sportivo (Utente registrato)
Parti interessate e interessi	Sportivo: vuole tenere traccia delle proprie attività e cancellare impegni futuri in caso di imprevisti.
Pre-condizioni	Lo sportivo ha effettuato il login al sistema.
Garanzie di successo	Lo stato della prenotazione viene modificato e, in caso di annullamento, la struttura torna nuovamente disponibile per altri sportivi.
Scenario principale di successo	1. Lo Sportivo accede alla propria area personale. 2. Il Sistema recupera e mostra l'elenco e lo stato delle prenotazioni (sia future che passate) associate a quello specifico sportivo. 3. Lo Sportivo seleziona una prenotazione futura e richiede di annullarla. 4. Il Sistema richiede un'ulteriore conferma per procedere con l'annullamento. 5. Lo Sportivo conferma l'operazione. 6. Il Sistema aggiorna lo stato della prenotazione in "Annullata" e rende la struttura nuovamente disponibile per la data e la fascia oraria corrispondente. 7. Il Sistema mostra un messaggio di successo allo Sportivo.

Estensioni

***a.** In qualsiasi momento il sistema fallisce e ha un arresto improvviso.

1. L'admin riavvia il software e ripristina lo stato precedente del sistema.

2. Il sistema ripristina lo stato.

3a. Lo Sportivo tenta di annullare una prenotazione passata o attualmente in corso.

1. Il Sistema segnala che l'operazione non è consentita per eventi già iniziati o trascorsi.

2. Lo Sportivo torna alla visualizzazione dell'elenco.

5a. Lo Sportivo rifiuta la conferma di annullamento.

1. Il Sistema interrompe l'operazione, mantenendo la prenotazione attiva e confermata.

2. Lo Sportivo torna alla visualizzazione dell'elenco.

Modello di dominio:

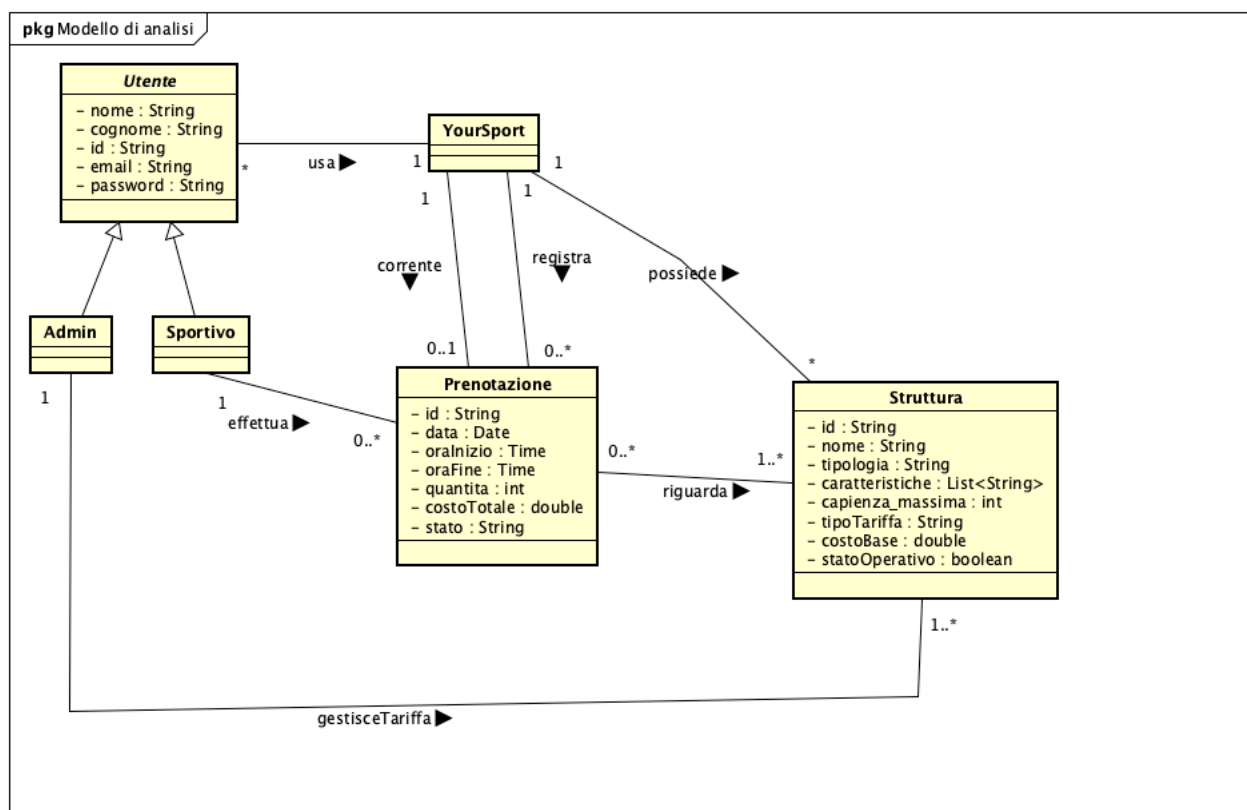
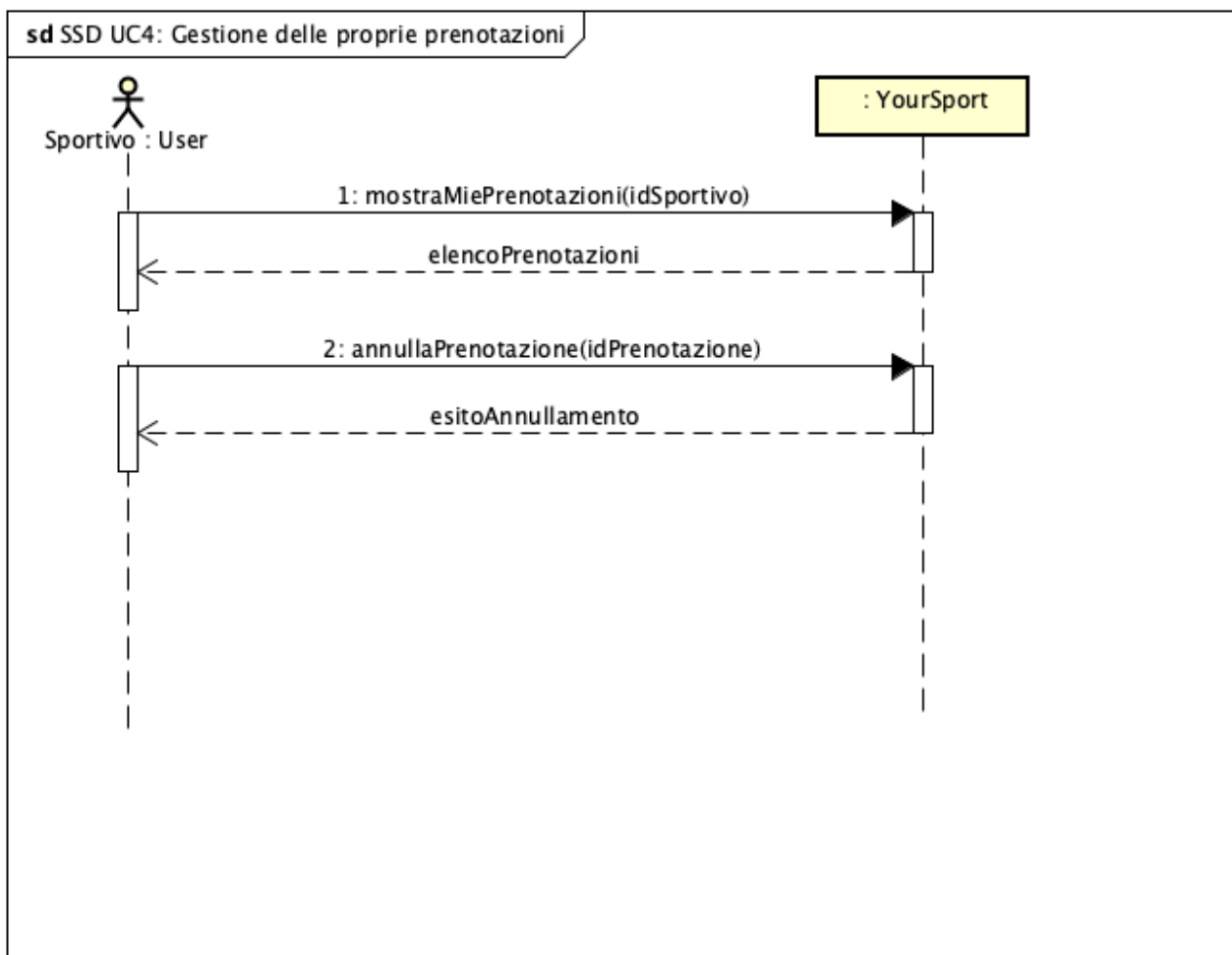


Diagramma di sequenza di sistema:



Contratti delle operazioni:

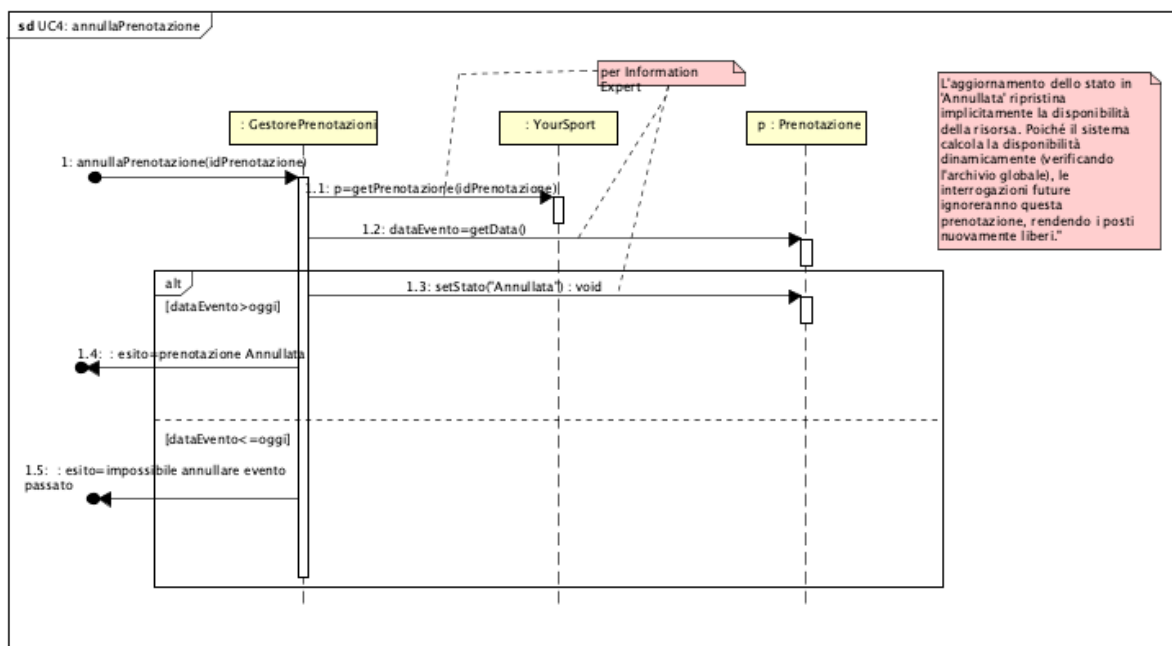
-UC4

Operazioni	annullaPrenotazione(idPrenotazione: String)
Riferimenti	UC4: Gestione delle proprie prenotazioni
Pre-condizioni	La prenotazione con idPrenotazione esiste nel sistema e la sua data/ora di inizio è successiva al momento attuale (è futura).

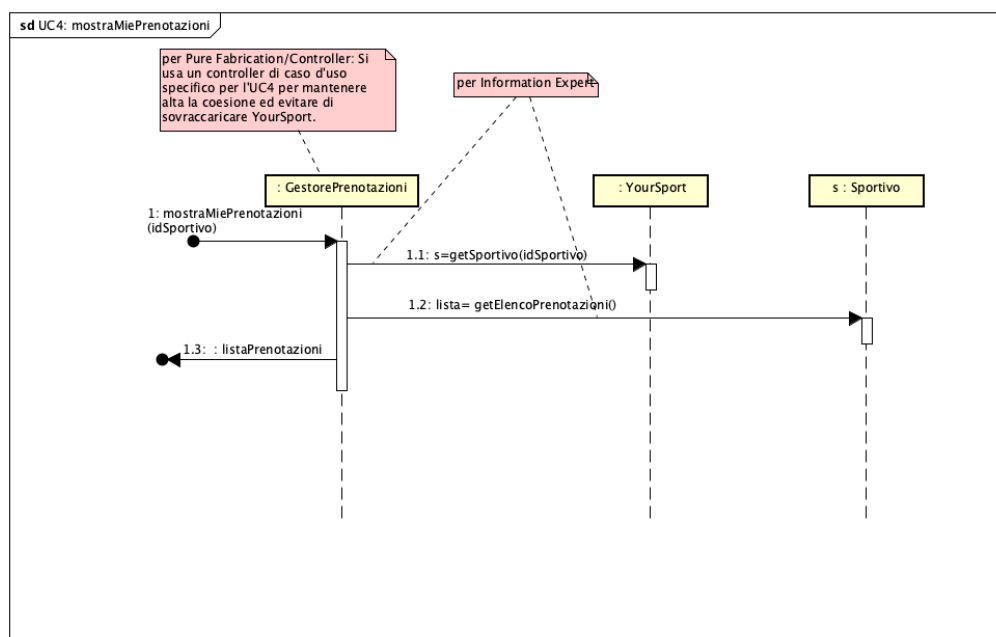
Post-condizioni	- L'attributo stato dell'istanza Prenotazione corrispondente è stato modificato in "Annullata" -La disponibilità della risorsa/struttura associata a quella prenotazione è stata ripristinata per quella data e fascia oraria
-----------------	---

Diagrammi di sequenza:

-annullaPrenotazione



-mostraMiePrenotazioni



Testing:

Test 1: Visualizzazione delle Proprie Prenotazioni

- **Descrizione:** Simuliamo l'utente loggato che accede alla propria area personale e chiede al sistema di visualizzare l'elenco delle prenotazioni da lui effettuate in precedenza.
- **Scopo:** Verificare che il nuovo controller (Pure Fabrication) `GestorePrenotazioni` comunichi correttamente con il Singleton `YourSport` per recuperare l'oggetto `Sportivo` corretto e ne restituisca la lista privata delle prenotazioni.
- **Realizzazione:** Il test prevede una fase di *setup* in cui il sistema crea una prenotazione valida per l'utente. Successivamente, si chiama il metodo `mostraMiePrenotazioni(id)`. Si utilizzano asserzioni (`assertNotNull`, `assertFalse`) per garantire che la lista ritornata non sia nulla o vuota, e un `assertEquals` finale per confermare che l'ID della prenotazione dentro la lista coincida esattamente con quello appena generato.

Test 2: Annullamento di una Prenotazione Futura (Happy Path)

- **Descrizione:** Simuliamo l'utente che decide di cancellare una sua prenotazione relativa a una data futura, fornendo al sistema il codice ID corretto.
- **Scopo:** Controllare il rispetto della guardia [`dataEvento > oggi`] nel blocco *ALT* dell'UC4, verificando che il sistema accetti l'annullamento e applichi la post-condizione, ovvero l'aggiornamento dello stato dell'oggetto.
- **Realizzazione:** Viene creata una prenotazione fittizia impostata per due giorni nel futuro. Si invoca il metodo `annullaPrenotazione()` passandone l'ID. Tramite `assertEquals` si verificano due cose fondamentali: che il messaggio di ritorno sia "prenotazione annullata" e, soprattutto, che l'attributo stato dell'oggetto `Prenotazione` in memoria sia effettivamente mutato in "Annullata".

Test 3: Tentativo di Annullamento con ID Inesistente (Flusso Alternativo)

- **Descrizione:** Simuliamo l'utente che, per errore o per manipolazione, inserisce nel sistema un codice ID di prenotazione non valido o non registrato nell'archivio.
- **Scopo:** Verificare la robustezza dell'implementazione e la corretta gestione degli input errati, assicurandosi che il sistema non vada in crash (es. `NullPointerException`) ma risponda in modo controllato.
- **Realizzazione:** Il test invoca il metodo `annullaPrenotazione("PRN-FALSO")`. Utilizzando un `assertEquals`, si verifica che il controller intercetti la mancanza dell'oggetto e restituisca immediatamente la stringa di cortesia: "errore: prenotazione inesistente".

